

Portal — Complete Developer Guide

The portal is the staff-facing admin dashboard. It is a separate Next.js application from the player frontend, sharing the same backend API. Staff use it to look up survey data (party sizes) collected during sessions.

Table of Contents

1. [Project Structure](#)
 2. [How the Portal Talks to the Backend](#)
 3. [Layer 1 — Root Layout](#)
 4. [Layer 2 — Survey Page \(survey/page.tsx\)](#)
 - [State Variables](#)
 - [Derived State \(computed values\)](#)
 - [Search Logic \(handleSearch\)](#)
 - [Reset Logic](#)
 - [Render Structure](#)
 5. [Layer 3 — Components \(components.tsx\)](#)
 6. [Layer 4 — Services](#)
 - [inputValidator.ts](#)
 - [surveyDatafetchService.ts](#)
 - [csvExportService.ts](#)
 - 6b. [Quick Export Page \(export/page.tsx\)](#)
 7. [Environment & Configuration](#)
 8. [Testing](#)
 9. [Adding New Features](#)
-

1. Project Structure

```
portal/
├─ Dockerfile           # Production multi-stage distroless image
├─ dockerfile.dev       # Development alpine image (hot reload)
├─ next.config.ts       # Next.js config (empty – no custom options currently)
├─ vitest.config.ts     # Test runner config – jsdom environment, globals,
coverage
├─ tsconfig.json        # TypeScript config
├─ package.json         # Portal-specific dependencies
├─
├─ src/
│  └─ app/
│     ├── layout.tsx    # Root HTML shell – sets page title and favicon
│     └── page.tsx      # Splash/auth screen – "Unauthorized Use Prohibited" gate,
routes to /menu
│  │  ├── components.tsx # All reusable UI components
│  │  ├── globals.css    # Global styles
│  │  └── menu/
│  │     └── page.tsx    # Navigation hub – links to /survey, /route, /export
│  │  └── survey/
│  │     └── page.tsx    # Survey data search tool (session ID / date range)
│  │  └── route/
│  │     └── page.tsx    # Route data placeholder – "Coming Soon"
```

```

|   |   |   └─ export/
|   |   |       └─ page.tsx      # Quick Export Tool – CSV download by reporting period
|   |   └─ services/
|   |       └─ csvExportService.ts      # Formats survey/route rows and triggers CSV
download
|   |       └─ inputValidator.ts        # Validation and sanitization for search inputs
|   |       └─ routeDataFetchService.ts # Fetch wrappers for the 10 route-stat
endpoints
|   |       └─ surveyDatafetchService.ts # Fetch wrappers for the two survey search API
calls
|
└─ testing/
    └─ setup.ts                        # Vitest global setup – mocks fetch,
localStorage, next/navigation
    └─ surveyPortalStructure.test.tsx
    └─ surveyPortalFunction.test.tsx
    └─ exportTool.test.tsx
    └─ inputValidator.test.tsx
    └─ layout.test.tsx
    └─ portalMenu.test.tsx
    └─ splashPage.test.tsx
    └─ UAT/
        └─ ExportSearchResults.test.tsx

```

The portal uses **Next.js App Router routing**. Each sub-directory under `app/` is a distinct URL route. Navigation between pages uses `useRouter().push(...)` — there is no shared state across pages.

2. How the Portal Talks to the Backend

The portal communicates with the backend via the same Express API as the player frontend. It uses **no authentication** — all portal endpoints are unauthenticated by design (internal staff tool).

API base URL is controlled by the environment variable `NEXT_PUBLIC_API_URL` :

```

// surveyDatafetchService.ts
const API_BASE = process.env.NEXT_PUBLIC_API_URL ?? "";

```

- In **development**: `NEXT_PUBLIC_API_URL=http://localhost:4000` (set in `docker-compose.dev.yml`)
- In **production**: `NEXT_PUBLIC_API_URL=""` — empty string, so requests go to `/api/...` relative to the portal's own origin. Nginx on port 3001 intercepts `/api/` and proxies it to the backend container.

Why empty string in production? Nginx handles the routing. Requests to `http://host:3001/api/...` are intercepted by the `3001` server block in `nginx.conf` and forwarded to `http://backend:4000` . The portal never needs to know the backend's address directly.

3. Layer 1 — Root Layout

File: [portal/src/app/layout.tsx](#)

Sets `<title>` to "Pollinator Habitat Admin Portal" and links the favicon. Renders a standard `<html lang="en"><body>{children}</body></html>` shell. No global components, no nav bar, no state.

4. Layer 2 — Survey Page (`survey/page.tsx`)

File: [portal/src/app/page.tsx](#)

The entire portal UI is one React component: `PortalPage` . It is marked "use client" — all logic runs in the browser, not the server. It renders as an HTML `<table>` layout (rows and cells for the search form and results).

State Variables

Variable	Type	Initial value	Purpose
<code>sessionId</code>	<code>string</code>	<code>""</code>	The text in the Session ID input
<code>startDate</code>	<code>string</code>	Yesterday (YYYY-MM-DD)	Start date for date-range search
<code>endDate</code>	<code>string</code>	Today (YYYY-MM-DD)	End date for date-range search
<code>searchResults</code>	<code>SurveyDataRow[]</code>	<code>[]</code>	Rows shown in the results table
<code>loading</code>	<code>boolean</code>	<code>false</code>	Disables the Search button during a fetch
<code>error</code>	<code>string null</code>	<code>null</code>	Error message shown in the debug row
<code>isRangeEnabled</code>	<code>boolean</code>	<code>false</code>	Whether the End Date input is active

`startDate` and `endDate` initialize lazily using `() => new Date().toISOString().slice(0, 10)` — so they always reflect the current day at component mount time, not at module import time.

Derived State (computed values)

These are plain variables (not `useState`) recomputed on every render:

```
const hasDateCriteria =
  (isValidDateInput(startDate) && !isRangeEnabled)
  || (isRangeEnabled && isValidDateInput(endDate) && isValidDateInput(startDate));

const hasSessionIdInput = sessionId.trim() !== "";
const hasSessionIdCriteria = isValidSessionIdInput(sessionId);

// Session ID takes priority if any text is in the session ID field
const useDateCriteria = !hasSessionIdInput && hasDateCriteria;
const canSearch = hasSessionIdInput ? hasSessionIdCriteria : hasDateCriteria;

const hasFormContent = sessionId !== "" || startDate !== "" || endDate !== "" ||
```

```
isRangeEnabled;  
const hasTableContent = searchResults.length > 0;
```

Priority rule: If the user has typed anything in the session ID field, session ID mode is active and the date inputs are ignored — even if the date inputs are also filled. This prevents ambiguous searches.

Search Logic (`handleSearch`)

```
handleSearch()  
|  
| — if !canSearch → set error "Invalid search criteria", return  
|  
| — if useDateCriteria (no session ID input, valid date)  
|   end = isRangeEnabled ? endDate : startDate ← single-date search sets both to  
startDate  
|   fetchSurveyDataByDateRange(startDate, end)  
|   → POST /api/get-child-adult-data-by-start-end-date  
|   → returns SurveyDataRow[] (one row per PlayerSession)  
|  
| — else (session ID mode)  
|   fetchChildrenAdultsBySessionId(sessionId)  
|   → POST /api/get-children-adults  
|   → aggregates all PlayerSession rows for that session  
|   → returns a single SurveyDataRow (totals for the session)
```

Both paths set `loading: true` before the call and `loading: false` in `finally`. Errors set the `error` state and clear `searchResults`.

Single-date search: When `isRangeEnabled` is `false`, both `startDate` and `endDate` are passed as the same value (`startDate`). The backend endpoint treats equal start/end dates as "all sessions on this exact day".

Reset Logic

There are two independent reset actions:

Function	What it clears
<code>handleSearchReset</code>	<code>sessionId</code> , <code>startDate</code> , <code>endDate</code> , <code>isRangeEnabled</code> , <code>error</code>
<code>handleTableReset</code>	<code>searchResults</code> , <code>error</code>

Reset buttons are disabled when there is nothing to clear (`!hasFormContent` and `searchResults.length === 0` respectively).

Render Structure

The page renders as an HTML `<table>` with `<tbody>` rows:

```
<tr class="table_row">          → Logo image (colspan 2)  
<tr class="header_row">        → "Pollinator Habitat Admin Portal" heading  
<tr class="search_header_row"> → Two column headers: "Search by Date" | "Search by  
Session ID"
```

```

<tr class="search_criteria_row">
  Left cell:
    - Start Date label + DateInput
    - End Date label + DateInput (disabled when isRangeEnabled=false)
    - "Date Range" checkbox (controls isRangeEnabled)
  Right cell:
    - "9 Digit Session ID" label + SessionIdInput
<tr class="search_button_row"> → Search button (colspan 2, disabled during loading)
<tr class="reset_buttons_row"> → Clear search | Clear table (independent buttons)
<tr class="debug_row">
  If error → <span role="alert" data-testid="search-error">{error}</span>
  Else → "Search is valid: true/false" (aria-hidden, visible debug aid for devs)
<tr class="table_row"> → SearchResultsTable (colspan 2)

```

The `debug_row` doubles as a user error display and a developer search-validity indicator. When no error is set, it shows the current value of `canSearch` — useful during development to verify validation logic without inspecting state.

5. Layer 3 — Components (`components.tsx`)

File: [portal/src/app/components.tsx](#)

All UI components are defined here and exported for use by `page.tsx`. Marked `"use client"`.

LogoImage

Renders ``. No props.

PortalHeader({ headerText })

Renders `<span class="portal_header">`. Used for the main page title.

SearchHeader({ headerText })

Renders `<h2 class="search_header">`. Used for the two column headers above the search inputs.

PortalText({ labelText })

Renders `<h1 class="portal_text">`. Used as field labels (e.g. "Start Date", "9 Digit Session ID").

InteractiveButton({ buttonText, onClick, disabled })

The primary action button. Renders `<button class="interactive_button">`. Used for the Search button.

ResetButton({ buttonText, onClick, disabled })

Secondary button. Renders `<button class="Reset-Button">`. Used for the two clear buttons.

DateInput({ value, onChange, disabled, dataTestId })

Wraps a native `<input type="date">`. Passes the raw YYYY-MM-DD string directly through `onChange`. The `disabled` prop disables the End Date input when `isRangeEnabled` is false.

`SessionIdInput({ value, onChange, onSubmit, dataTestId })`

Wraps a text input for the session ID. Key behaviors:

- `maxLength={9}` — prevents typing more than 9 characters
- `onChange` calls `sanitizeSessionIdInput(e.target.value)` before setting state — strips all non-digit characters on every keystroke
- Placeholder: "Ex: 184293921"
- `aria-label="Session ID Input"` for accessibility

`SearchResultsTable({ searchResults })`

Renders a `<table class="results_table">` with columns: **Session ID | Date | Adults | Children | Total**.

Empty state: if `searchResults` is empty, renders a placeholder row `{ sessionId: "No results", date: "--/--/----", numAdults: "--", numChildren: "--" }`.

Total column: computed as `numAdults + numChildren`. If either value is not a valid number, shows `--`.

The component accepts `searchResults` typed as:

```
{ sessionId: string; date: string; numAdults?: number | string; numChildren?: number | string }[]
```

Null/undefined/empty string values for `numAdults` and `numChildren` display as `--`.

6. Layer 4 — Services

`inputValidator.ts`

File: <portal/src/services/inputValidator.ts>

Three pure functions. No side effects, no imports.

```
sanitizeSessionIdInput(input: string): string
```

Strips all non-digit characters (`/\D/g`) and truncates to 9 characters. Called in `SessionIdInput.onChange` on every keystroke — prevents letters, symbols, and pasted non-numeric strings from ever entering the state.

```
isValidSessionIdInput(sessionId: string): boolean
```

Returns `true` if and only if `sessionId.length === 9`. Does not check whether the session actually exists in the database — that is the backend's job. Length-9 is the only format constraint enforced client-side.

```
isValidDateInput(date: string): boolean
```

Returns `true` if:

1. `date` is non-empty
2. Matches `/^20\d{2}-\d{2}-\d{2}$/` — must be a `20xx` year in YYYY-MM-DD format
3. `new Date(date)` parses to a valid date (not NaN)

The year prefix `20` is hardcoded in the regex — dates before 2000 are rejected. This is intentional; the app will never have sessions before 2000.

surveyDatafetchService.ts

File: [portal/src/services/surveyDatafetchService.ts](#)

Exports two async functions and the `SurveyDataRow` type.

```
type SurveyDataRow = {
  sessionId: string;
  date: string;
  numAdults?: number | string;
  numChildren?: number | string;
};
```

This is the shape both fetch functions return and that `SearchResultsTable` consumes.

fetchSurveyDataByDateRange(startDate, endDate): Promise<SurveyDataRow[]>

Calls `POST /api/get-child-adult-data-by-start-end-date` with:

```
{ "startDate": "2026-01-01", "endDate": "2026-03-31" }
```

Response shape from backend:

```
{
  "success": true,
  "dataload": [
    { "sessionId": 18429, "createdAt": "2026-01-15T...", "numAdults": 2,
    "numChildren": 3 },
    ...
  ]
}
```

Mapping:

- `sessionId` → `String(r.sessionId)` (backend sends a number, table expects a string)
- `date` → `new Date(r.createdAt).toISOString().slice(0, 10)` (ISO 8601 → YYYY-MM-DD)
- `numAdults` / `numChildren` → pass through, or `"--"` if null

Error handling:

1. Checks `Content-Type` header — if not `application/json` or body starts with `<`, throws `"server returned an invalid response"`. This catches cases where Nginx returns an HTML error page when the backend is down.
2. If `JSON.parse` fails → throws `"invalid response from server"`

```
3. If !res.ok || !data.success || !Array.isArray(data.dataLoad) → throws data.message  
?? data.error ?? "Search failed"
```

fetchChildrenAdultsBySessionId(sessionId): Promise<SurveyDataRow[]>

Calls POST /api/get-children-adults with:

```
{ "sessionId": 18429 }
```

Note: sessionId is coerced to a Number before sending — the backend expects a number, not a string.

Response shape from backend:

```
{  
  "success": true,  
  "sessionId": "2026-01-15",  
  "data": [  
    { "playerId": 1, "numChildren": 3, "numAdults": 2 },  
    { "playerId": 2, "numChildren": 0, "numAdults": 4 }  
  ]  
}
```

Aggregation: Unlike the date-range function which returns one row per session, this function **sums** all numAdults and numChildren across all PlayerSession rows for the session and returns a **single** SurveyDataRow :

```
const totalAdults = items.reduce((sum, item) => sum + (item.numAdults ?? 0), 0);  
const totalChildren = items.reduce((sum, item) => sum + (item.numChildren ?? 0), 0);  
return [{ sessionId, date: data.sessionDate ?? "--", numAdults: totalAdults,  
  numChildren: totalChildren }];
```

If data.data is an empty array, returns [] (no rows — the session exists but no party-size surveys were submitted).

Same Content-Type and JSON parse error handling as the date-range function.

csvExportService.ts

File: <portal/src/services/csvExportService.ts>

Two exported functions. No side effects beyond triggering a browser file download.

```
exportToCsv(filename: string, rows: SurveyDataRow[]): void
```

Accepts a filename string and an array of SurveyDataRow objects (the same shape returned by surveyDatafetchService). Builds a CSV and triggers an immediate browser download.

Column order: Session ID, Date, Adults, Children, Total

Adults and Children are coerced from the row's numAdults / numChildren fields: null and "" are treated as 0 . Total is computed as Adults + Children — it is never read from the API response.


```
exportRouteStatsToCsv(filename: string, rows: RouteStatRow[]): void
```

Same download mechanism as `exportToCsv`, but for route stat rows (`RouteStatRow[]` from `routeDataFetchService`).

Column order: `Metric`, `Value` — the metric label is quoted to handle any commas in the text.

Download mechanism (both functions):

1. Joins headers and rows into a newline-delimited CSV string
2. Wraps in a `Blob` with `type: "text/csv;charset=utf-8;"`
3. Creates an object URL via `URL.createObjectURL`
4. Appends a temporary `<a>` element to `document.body`, sets `href` and `download`, calls `.click()`
5. Removes the element and revokes the object URL immediately after

This pattern is compatible with all modern browsers and requires no server round-trip.

`routeDataFetchService.ts`

File: <portal/src/services/routeDataFetchService.ts>

Fetches the 10 route engagement metrics from the backend in two modes: **date range** and **session ID**. All 10 requests fire in parallel per query using `Promise.all`.

```
export type RouteStatRow = { metric: string; value: number | string };
```

The shape returned to the UI — `metric` is the human-readable label, `value` is the result (or `"--"` / `"Error"` on failure).

```
fetchRouteStatsByDateRange(startDate: string, endDate: string):  
Promise<RouteStatRow[]>
```

POSTs `{ startDate, endDate }` to each of the 10 `/api/route-stats/*-by-date-range` endpoints in parallel. Individual endpoint failures return `{ metric: label, value: "Error" }` rather than failing the whole batch.

```
fetchRouteStatsBySession(sessionId: string): Promise<RouteStatRow[]>
```

POSTs `{ sessionId: Number(sessionId) }` to each of the 10 `/api/route-stats/*-by-session` endpoints in parallel. Same per-item error handling.

See ROUTE_DATA_FETCH_SERVICE.md for full line-by-line detail.

6b. Quick Export Page (`export/page.tsx`)

File: <portal/src/app/export/page.tsx>

The Quick Export Tool page. Staff select a reporting period and a data type, then click "Export Data" to fetch and immediately download a CSV. No results table is shown — the export fires directly to the browser's

download prompt.

State

Variable	Type	Description
selectedPeriod	string null	The chosen reporting period. One of: "1month", "3months", "6months", "1year", "3years", "alltime". null if none selected.
selectedData	string null	The chosen data type. One of: "survey", "route". null if none selected.
loading	boolean	Disables the Export button during the fetch.
error	string null	Error message shown below the Export button.

Checkboxes within each group are mutually exclusive — selecting one deselects the previous.

Reporting Period Options

Label	State value	Start date
1 Month	"1month"	1 month ago
3 Months	"3months"	3 months ago
6 Months	"6months"	6 months ago
1 Year	"1year"	1 year ago
3 Years	"3years"	3 years ago
All Time	"alltime"	"2020-01-01" (hardcoded project epoch)

End date is always today. `periodToStartDate(period)` converts the selection to a `YYYY-MM-DD` string.

Data Type Options

Label	State value	Fetch function	Export function
Survey Report	"survey"	<code>fetchSurveyDataByDateRange</code>	<code>exportToCsv</code>
Route Report	"route"	<code>fetchRouteStatsByDateRange</code>	<code>exportRouteStatsToCsv</code>

Export Handler

`handleExport()` fires when the Export button is clicked. It calls `periodToStartDate(selectedPeriod)` to get the start date, sets the end date to today, then branches on `selectedData` to call the appropriate fetch + export pair. Errors are stored in `error` state and shown inline below the button. The button label switches to `"Exporting..."` while `loading` is true.

`canExport = selectedPeriod !== null && selectedData !== null` — both selections must be made before the button is enabled.

See [EXPORT_PAGE.md](#) for full line-by-line detail.

7. Environment & Configuration

Variable	Dev value	Prod value	Purpose
NEXT_PUBLIC_API_URL	http://localhost:4000	""	Backend base URL for fetch calls
NODE_ENV	development	production	Controls Next.js build mode

`NEXT_PUBLIC_` prefix means this value is inlined at **build time** by Next.js — it is not a runtime env var. In production, the empty string is baked into the JS bundle, so all API calls are relative (`/api/...`).

Changing the API URL requires a rebuild — you cannot swap it with a runtime variable after the image is built. This is a Next.js constraint for public env vars.

8. Testing

Test runner: Vitest with `jsdom` environment.

Run tests:

```
# From Pollinator-Habitat/  
npm run test:portal  
  
# From portal/  
npx vitest
```

Global Setup ([testing/setup.ts](#))

Runs before every test file. Does three things:

1. **Sets** `NEXT_PUBLIC_API_URL` to `http://localhost:4000` if not already set — prevents the fetch service from constructing malformed URLs in tests.
2. **Mocks** `next/navigation` — provides stub implementations of `useRouter` , `usePathname` , and `useSearchParams` . Required because `page.tsx` is a client component and Next.js navigation hooks throw outside the Next.js runtime.
3. **beforeEach / afterEach hooks:**
 - Replaces `window.localStorage` with a Vitest mock (getItem returns null, setItem/removeItem/clear are stubs)
 - Replaces `global.fetch` with a mock that returns `{ ok: true, json: async () => ({ data: 'mock data' }) }` — tests that need different fetch behavior must override this in the test itself
 - `afterEach` : calls `vi.clearAllMocks()` to reset all mock state between tests

Test Files

File	What it tests
<code>surveyPortalStructure.test.tsx</code>	Static rendering — logo, header, labels are present

<code>surveyPortalFunction.test.tsx</code>	Interactive behavior — search, reset, loading state, error display
<code>exportTool.test.tsx</code>	CSV export service tests
<code>inputValidator.test.tsx</code>	All three validator/sanitizer functions, edge cases
<code>layout.test.tsx</code>	Root layout renders without error
<code>portalMenu.test.tsx</code>	Portal menu/navigation rendering
<code>splashPage.test.tsx</code>	Portal splash page rendering
<code>UAT/ExportSearchResults.test.tsx</code>	Smoke test — PortalPage renders without crashing

See [TEST DOCUMENTATION.md](#) for full per-test documentation.

9. Adding New Features

Adding a new data view (e.g. Route Statistics)

1. **Create a fetch function** in `surveyDatafetchService.ts` (or a new service file) following the same pattern: check Content-Type, parse JSON, throw descriptive errors, return typed rows.
2. **Add validator logic** in `inputValidator.ts` if the new view has different input constraints.
3. **Add state** to `PortalPage` for the new view's results, loading, and error.
4. **Add UI** — either extend `page.tsx` with new table rows, or add a tab/section component. New reusable UI elements go in `components.tsx`.
5. **Add tests** — unit tests for the fetch function and validator, a render test for the new UI section.

The Route Statistics feature (10 engagement metrics) is fully specified in [ROUTE STATISTICS SPEC.md](#). The backend (20 endpoints) is fully implemented; the portal UI section is not yet started.

Adding a new component

Add it to `components.tsx`. Export it. Import it in `page.tsx` by name. The import line at the top of `page.tsx` is a named import from `"/components"` — add your new component name there.

Changing the backend API URL

For development: change `NEXT_PUBLIC_API_URL` in `docker-compose.dev.yml` under the `portal` service.

For production: the value is baked in as `""` (empty string) so Nginx routing handles it. To point at a different backend, change the environment variable in the compose file **before building the image** — a rebuild is required.

Not tracked by git — update when portal files change.